Solutions

FDE Tutorial Sessions, Week 4

Exercise 1

Exercise 1.1

a	b	~a	a & b	a | b	a ^ b
0	0	1	0	0	0
0	1	1	0	1	1
1	0	0	0	1	1
1	1	0	1	1	0

NOT - flips the bit

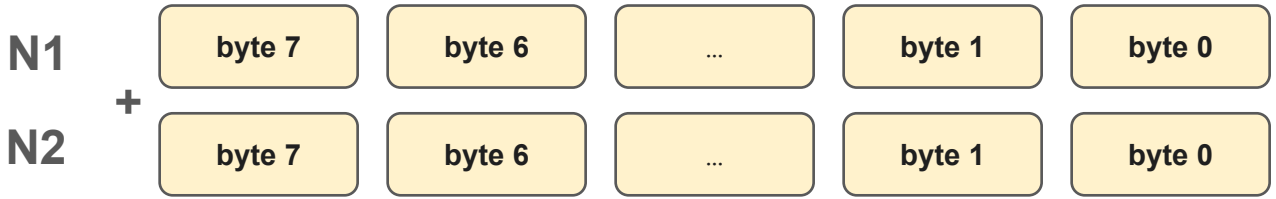
AND - 1 if both are 1

OR - 1 if at least one is 1

XOR - 1 if exactly one is 1

Exercise 1.2

One 64-bit **add** = 8 independent byte additions (but **only if the highest bit of every byte is 0** in both operands)



Per byte (should be valid for all 8 bytes in each number):

If both bytes ≤ 127

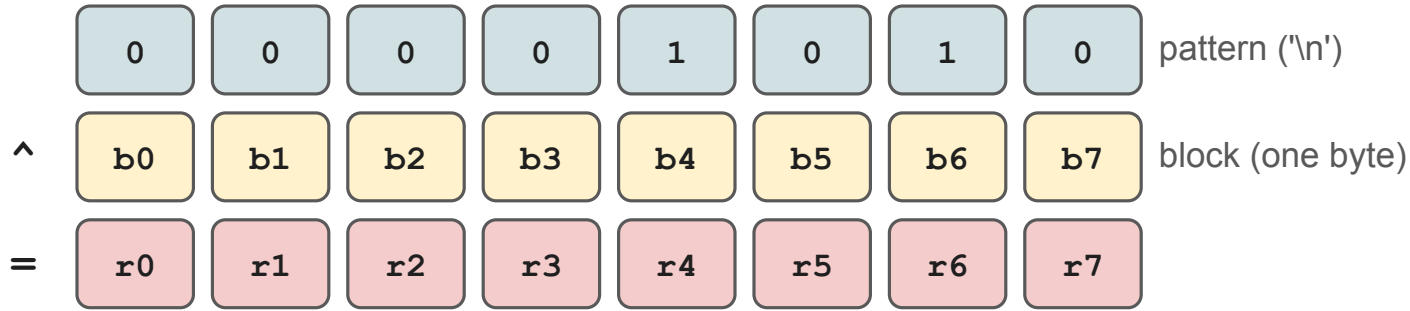
```
  0111 1111   (127)
+ 0111 1111   (127)
= 1111 1110   (254)
```

If highest bit set (>127):

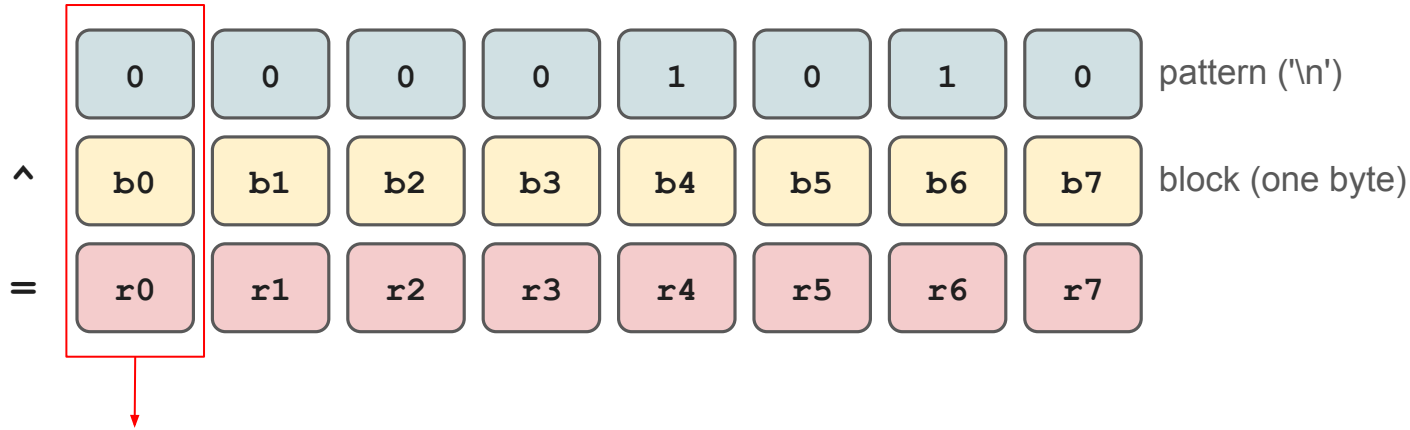
```
    1000 0000   (128)
+   1000 0000   (128)
= 1 0000 0000   (256)
```

the carry spills into the next byte

Exercise 1.3

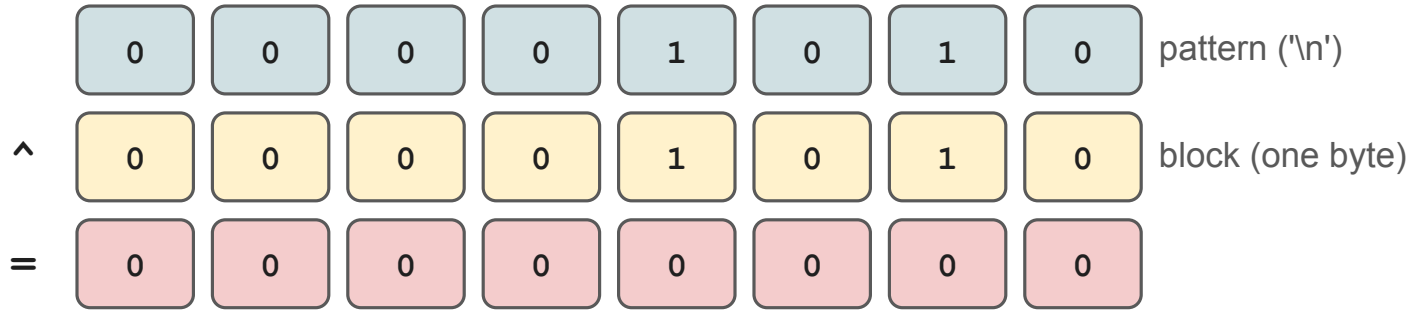

0 0 0 0 1 0 1 0
pattern ('\n')
^
b0 b1 b2 b3 b4 b5 b6 b7
block (one byte)
=
r0 r1 r2 r3 r4 r5 r6 r7

Exercise 1.3



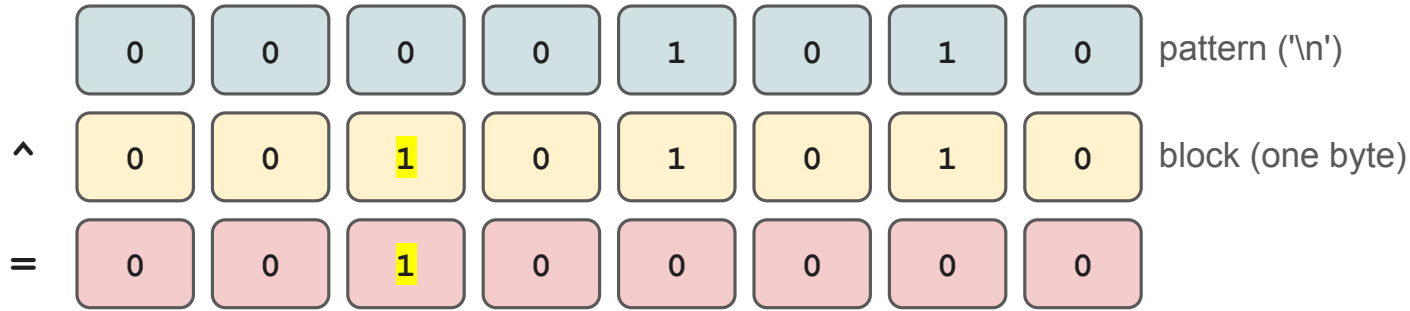
`0 ^ b0 = 0` *only if* `b0 = 0`

Exercise 1.3



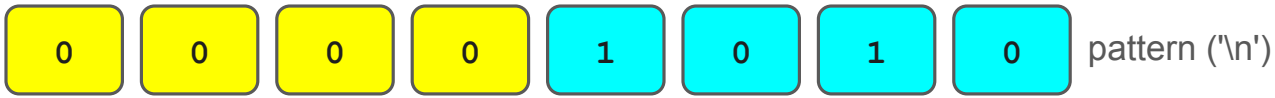
The byte matches the pattern → result is all 0

Exercise 1.3



The bits in the mismatched positions in the result are 1

Exercise 1.3

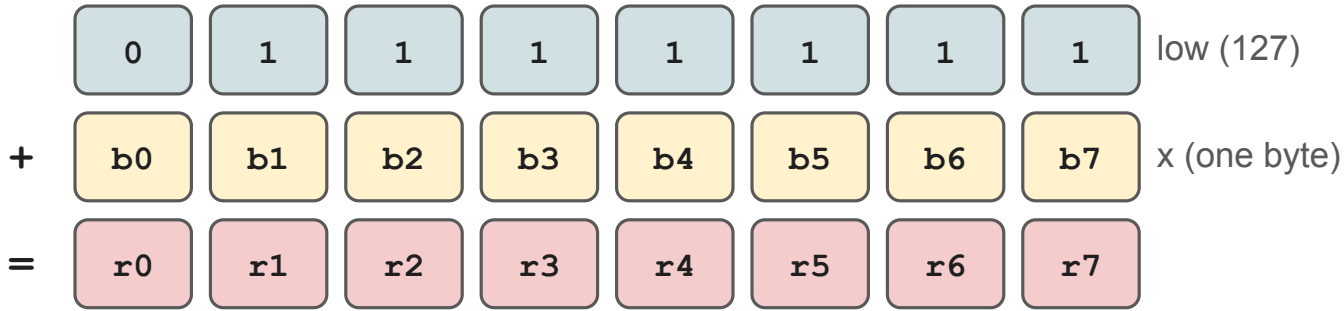


(binary) 00001010 → 0A *(hex)*

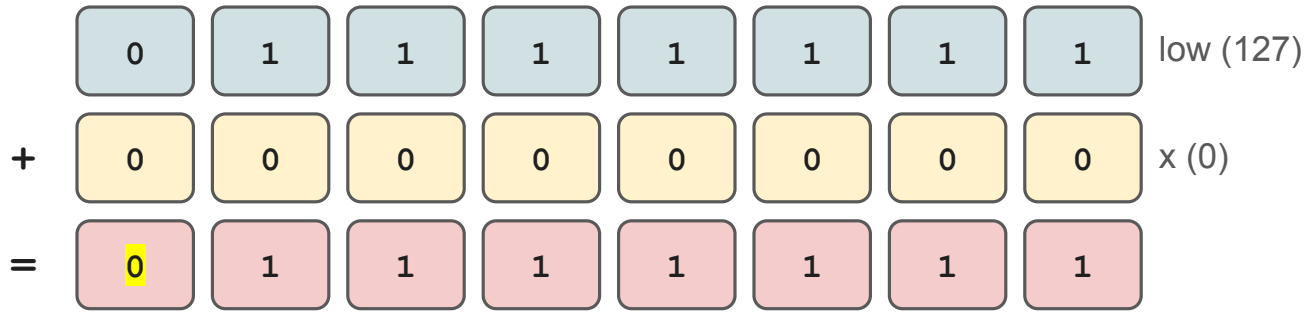
Applied to all 8 bytes:
0x0A0A0A0A0A0A0A0A

```
uint64_t pattern = 0x0A0A0A0A0A0A0A0A;
uint64_t all0iffNl = block ^ pattern;
```

Exercise 1.4*

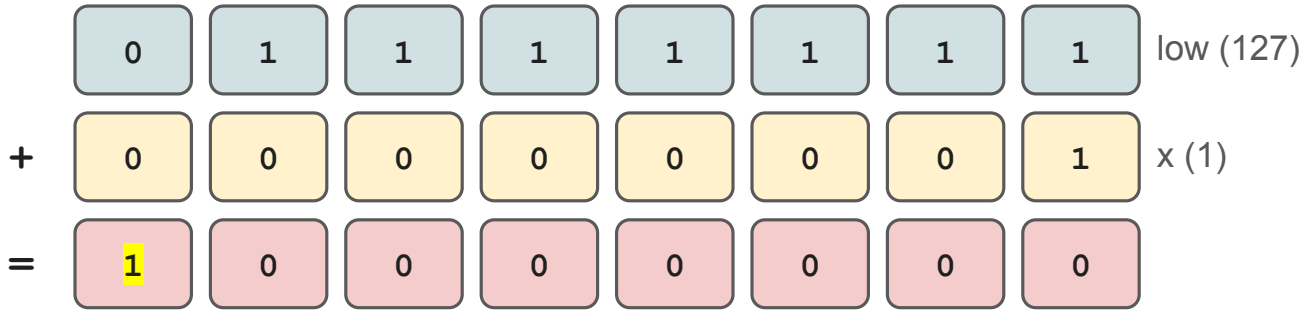

0 1 1 1 1 1 1 1
low (127)
+
b0 b1 b2 b3 b4 b5 b6 b7
x (one byte)
=
r0 r1 r2 r3 r4 r5 r6 r7

Exercise 1.4*



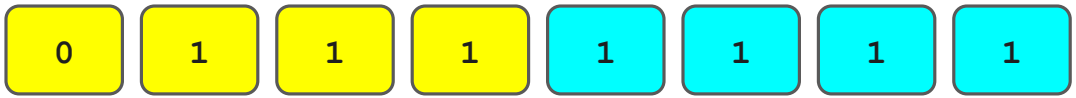
`x` = 0 → the highest bit is 0

Exercise 1.4*



`x = 1` → the highest bit is 1

Exercise 1.4*



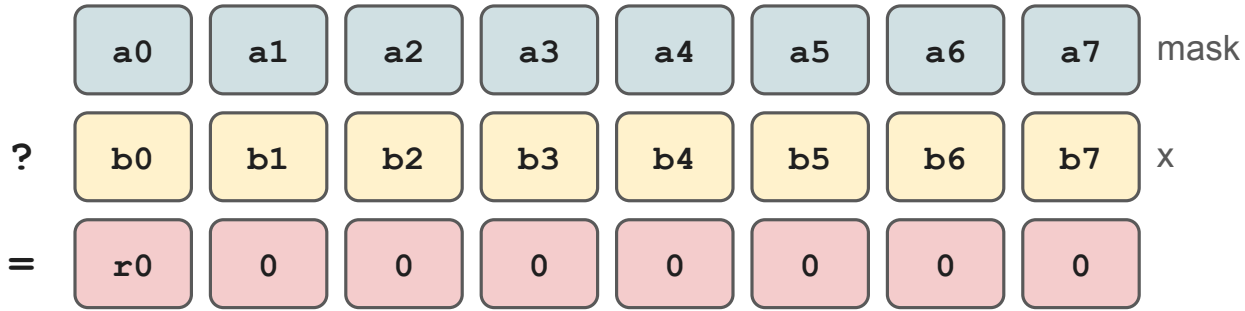
(binary) 01111111 → 7F *(hex)*

Applied to all 8 bytes:
0x7F7F7F7F7F7F7F7F

```
uint64_t low = 0x7F7F7F7F7F7F7F7F;
uint64_t highestBitSetIffNot0 = all0iffNl + low;
```

Exercise 1.5*

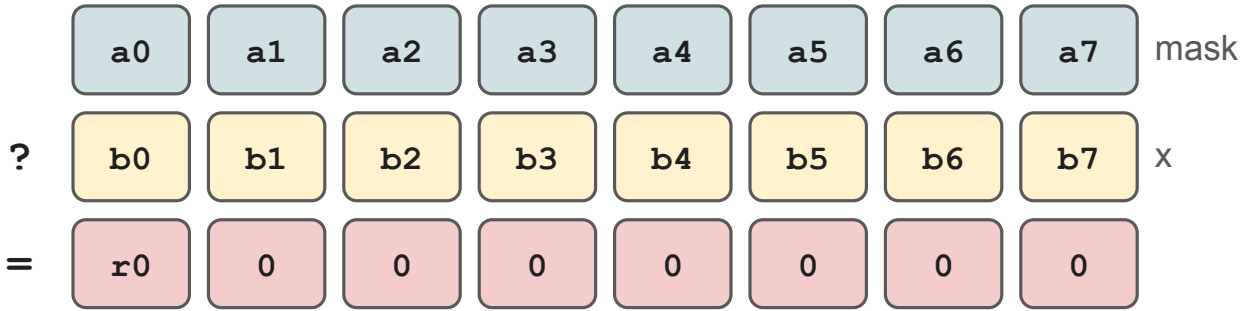
We want:



b0 = 1 → r0 = 0
b0 = 0 → r0 = 1

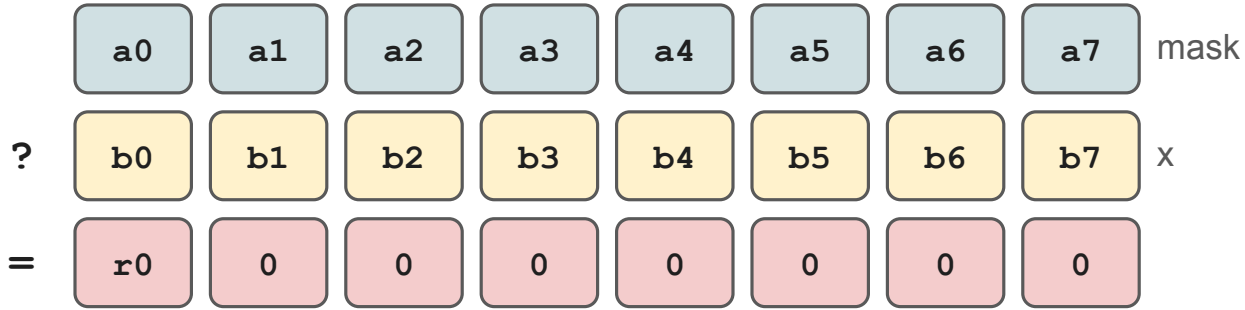
Exercise 1.5*

Subtask 1:



No matter the values of **b1, …, b7** → **r1, …, r7** are 0

Exercise 1.5*

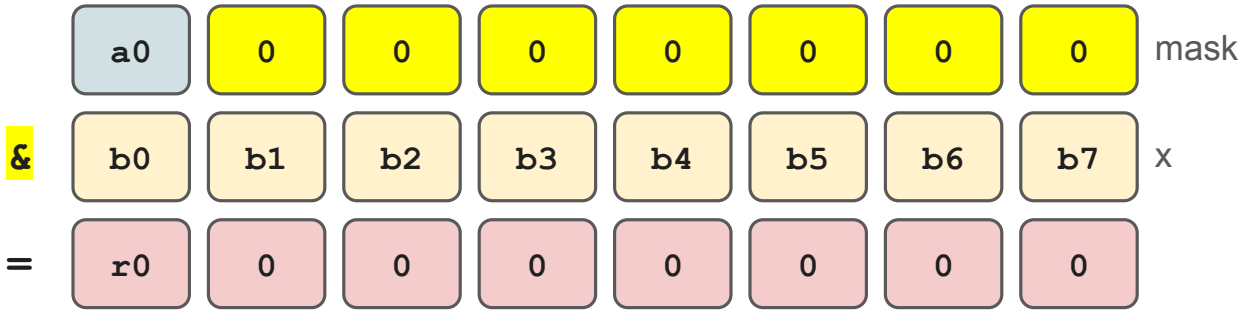


a	b	~a	a & b	a | b	a ^ b
0	0	1	0	0	0
0	1	1	0	1	1
1	0	0	0	1	1
1	1	0	1	1	0

only this depends on both a and b!

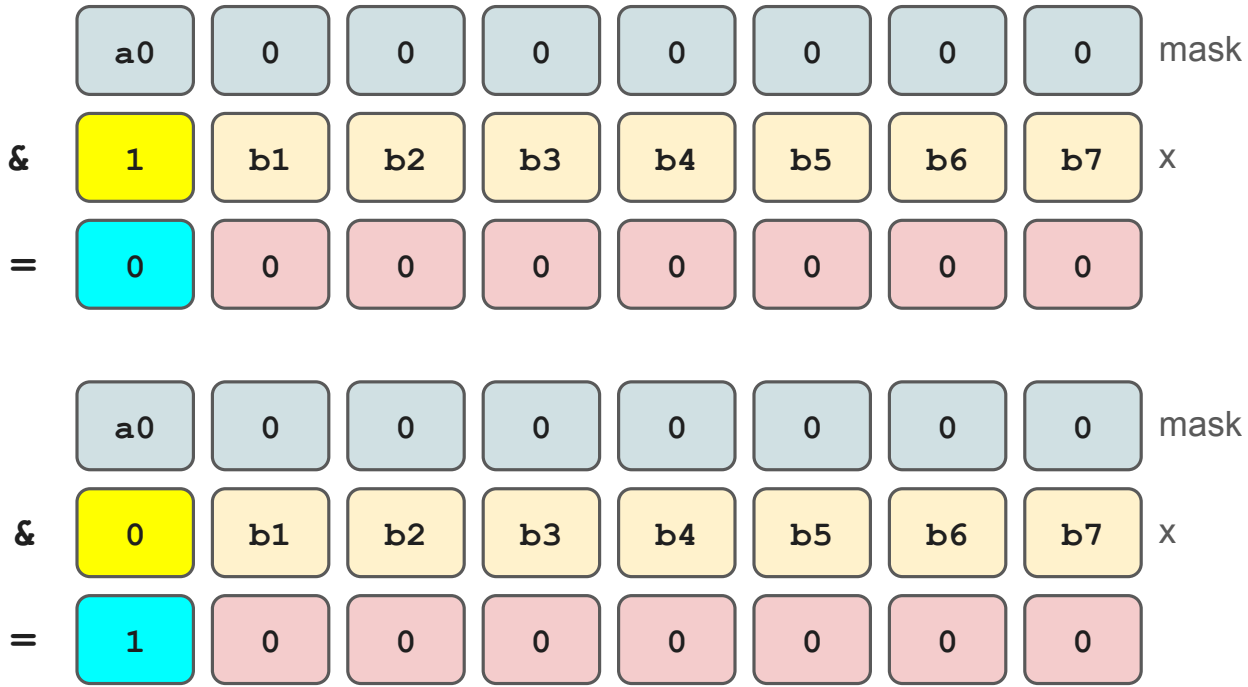
Exercise 1.5*

Subtask 1 solution: use AND, set bits to 0



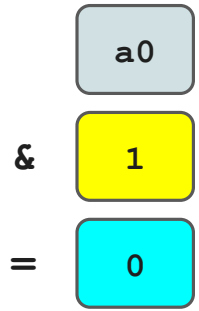
Exercise 1.5*

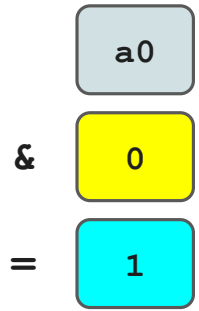
Subtask 2:



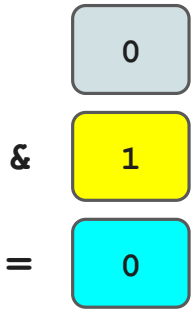
Exercise 1.5*

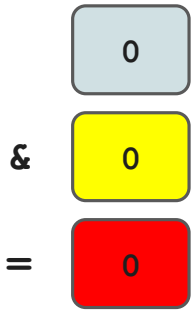
Goal:



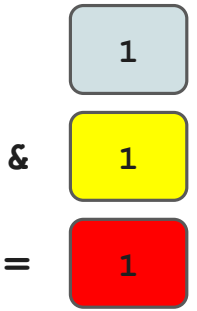


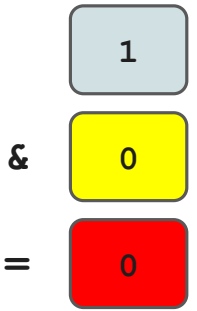
`a0 = 0`





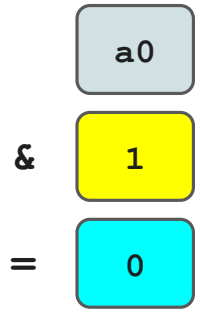
`a0 = 1`

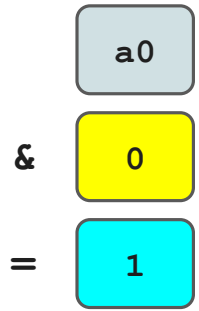




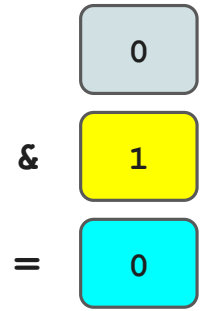
Exercise 1.5*

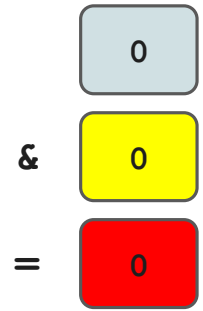
Goal:



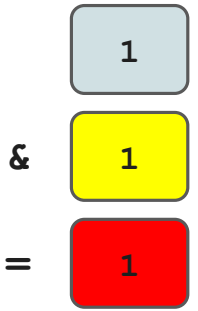


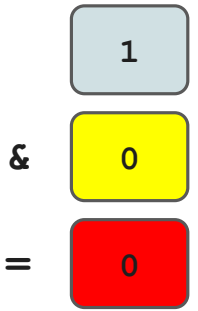
`a0 = 0`





`a0 = 1`

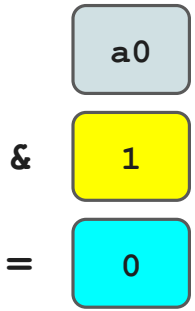


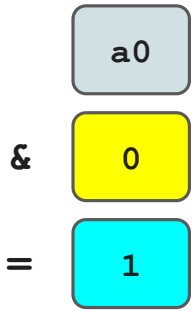


Exact opposite of what we want
→ use NOT!

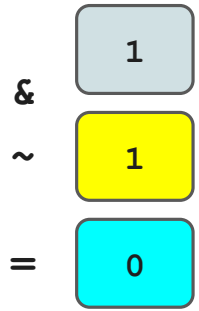
Exercise 1.5*

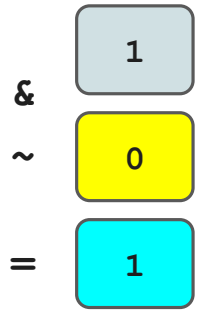
Goal:





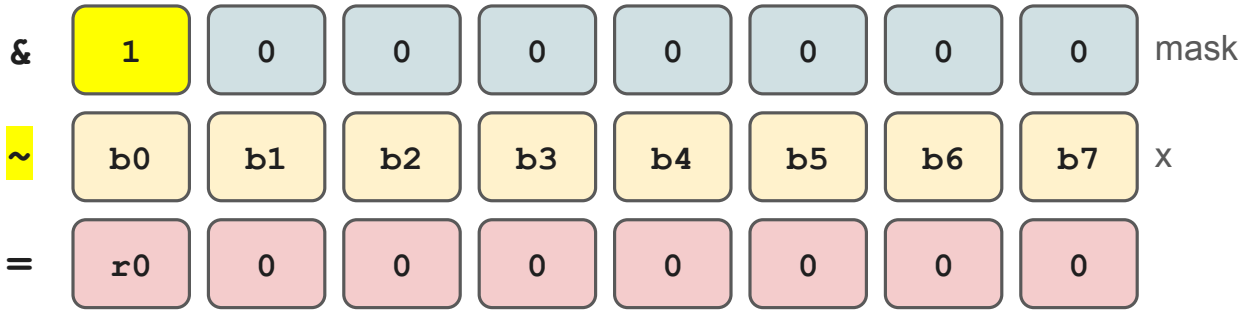
`a0 = 1`



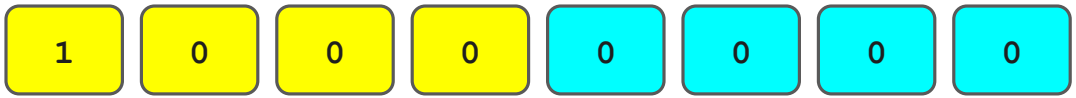


Exercise 1.5*

Subtask 2 solution: set bit to 1, use NOT



Exercise 1.5*



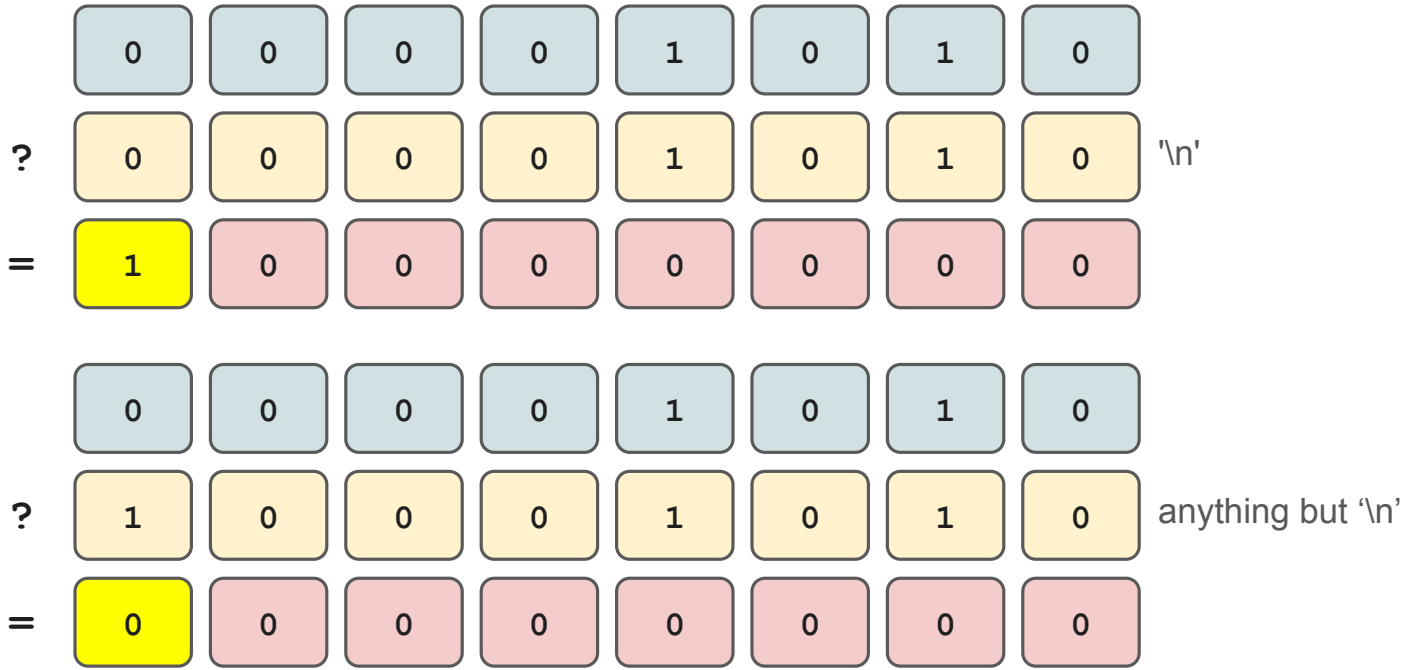
(binary) 10000000 → 80 *(hex)*

Applied to all 8 bytes:
0x8080808080808080

```
uint64_t high = 0x8080808080808080;
uint64_t highestSetIf0 = ~highestBitSetIffNot0 & high;
```

Exercise 1.6*

Goal: the 8-bytes-at-once newline search should also work when some bytes are > 127
Result: highest bit 1 in every byte that is ‘\n’, 0 everywhere else



Exercise 1.6*

Subtasks from the previous exercises (1.3 - 1.5):

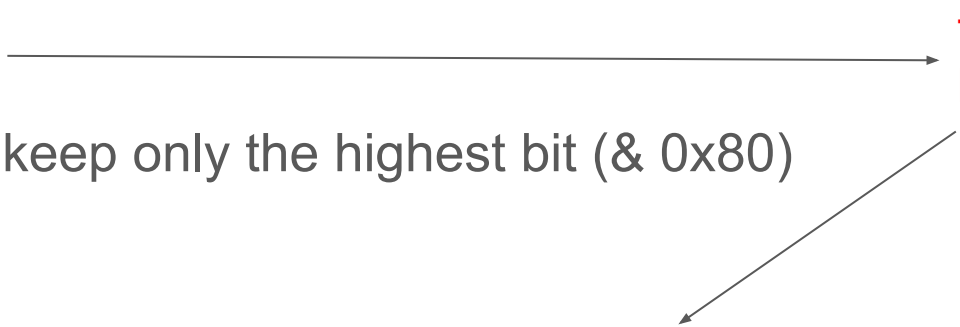
1. XOR with 0x0A (the code for ‘\n’)
2. add 0x7F
3. NOT, then keep only the highest bit (& 0x80)

this needs the highest bit to be 0, else overflow

Exercise 1.6*

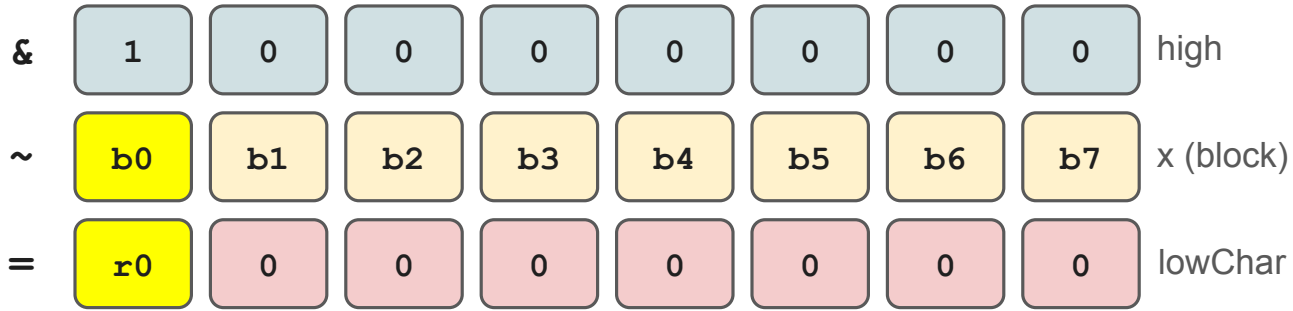
Subtasks from the previous exercises:

1. XOR with 0x0A (the code for ‘\n’)
2. add 0x7F
3. NOT, then keep only the highest bit (& 0x80)



Exercise 1.6*

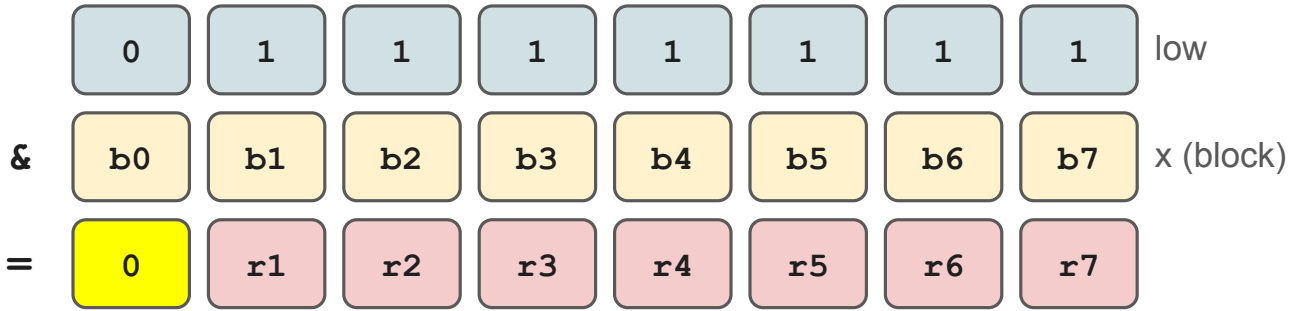
Subtask 1: set highest bit to 1 only if `x ≤ 127` (from 1.5)




```
uint64_t lowChar = ~block & high;
```

Exercise 1.6*

Subtask 2: set highest bit to 0, preserve everything else




```
block = block & low;
```

Exercise 1.6*

Now execute the steps from 1.3 - 1.5 on the new block:

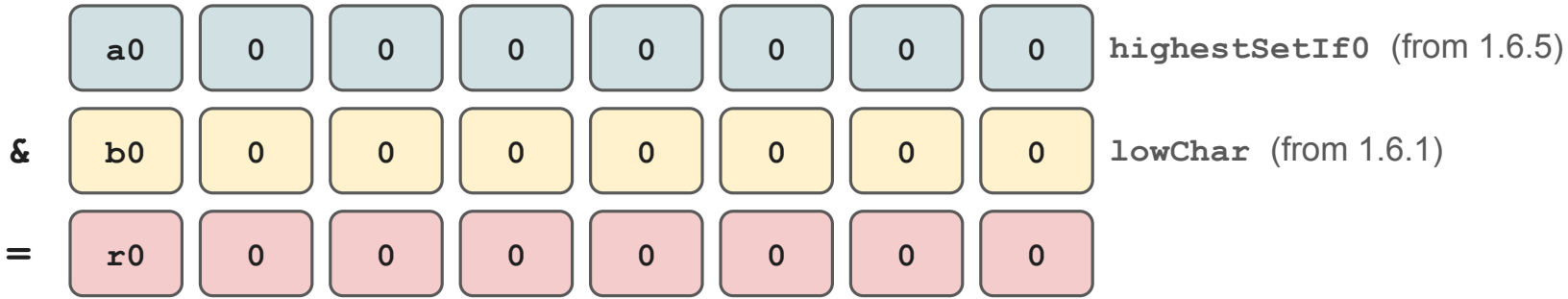
3. XOR with 0x0A (the code for ‘\n’)
4. add 0x7F
5. NOT, then keep only the highest bit (& 0x80)



not a problem anymore after subtask 2!

Exercise 1.6*

Subtask 6: keep a match only if the original byte was ≤ 127




```
uint64_t result = highestSetIf0 & lowChar;
```

1.7, 1.8 - skipped in the interest of time

Exercise 2

Exercise 2.1 (Hash Join Refresher)



- Build: O(|S|), probe: O(1) per row on average
- Total: O(|R| + |S|) instead of O(|R|·|S|)
- **Only for equality conditions (=)**
- Fits better in caches

Exercise 2.1

BUILD side in parallel → race conditions

- **std::unordered_set** can't handle several threads inserting at once → lost entries or a crash
- a **global sum**: threads overwrite each other's addition

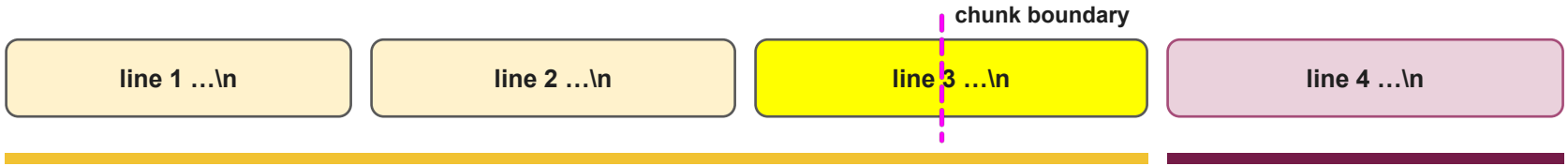
PROBE side in parallel → safe

- the hash table is only **read**
- every line item is independent → **embarrassingly parallel**
- as long as each thread keeps its **own local sum**

thread 1	thread 2	sum
read sum → 5		5
	read sum → 5	5
write 5 + 3		8
	write 5 + 2	7 (the +3 is lost)

Exercise 2.2

A line belongs to the chunk where it starts!



chunk 1: lines 1 – 3, finishes line 3 even past its end

chunk 2: starts at line 4

- **start**: a chunk not at 0 skips to the first line starting **at or after** its beginning
- **end**: process every line that starts **before** end, the last one up to its '\n'

Edge case: a line starting exactly at begin → search for '\n' from begin - 1, so it is not skipped

Exercise 2.2

```
std::size_t pos = begin;
if (begin > 0) {
    pos = begin - 1;
    skipLine<UseSwar>(data, size, pos);
}

std::int64_t sum = 0;
while (pos < end) {
    int orderkey = toInt(nextField<UseSwar>(data, size, pos));
    for (int i = 2; i <= 4; ++i) {
        nextField<UseSwar>(data, size, pos);
    }
    int quantity = toInt(nextField<UseSwar>(data, size, pos));
    if (urgentOrders.contains(orderkey)) {
        sum += quantity;
    }
    skipLine<UseSwar>(data, size, pos);
}
return sum;
```

Exercise 2.3*

```
...
std::vector<std::int64_t> results(numThreads, 0);
std::vector<std::thread> threads;
std::size_t chunkSize = (size + numThreads - 1) / numThreads;
for (unsigned i = 0; i < numThreads; ++i) {
    std::size_t begin = std::min(size, i * chunkSize);
    std::size_t end = std::min(size, begin + chunkSize);
    threads.emplace_back([&, i, begin, end] {
        results[i] = probeChunk<true>(data, size, begin, end, urgentOrders);
    });
}
for (std::thread& thread : threads) {
    thread.join();
}
return static_cast<double>(std::accumulate(results.begin(), results.end(), std::int64_t{0}));
```

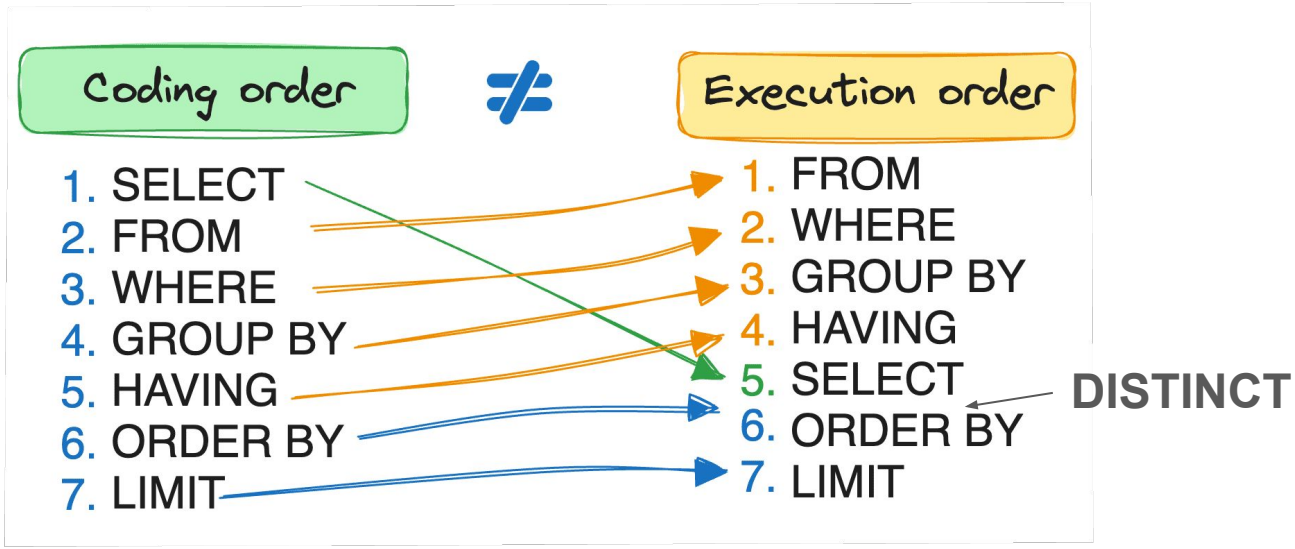
Exercise 3

Exercise 3*

SQL	GNU tools	seen in the last weeks
SELECT	`cut, awk` *(can compute)*	`cut -d'|' -f1,5`
FROM	`cat, zcat / xzcat, join` *(tools that read files)*	`join … <(…) <(…)`
WHERE	`grep, awk`	`grep EUROPE region.tbl`
GROUP BY	`(sort +) awk`	`awk '{ groups[$2] += $5 }'`
ORDER BY	`sort`	`sort -n -t'|' -k2`
LIMIT	`head, tail`	`| head`

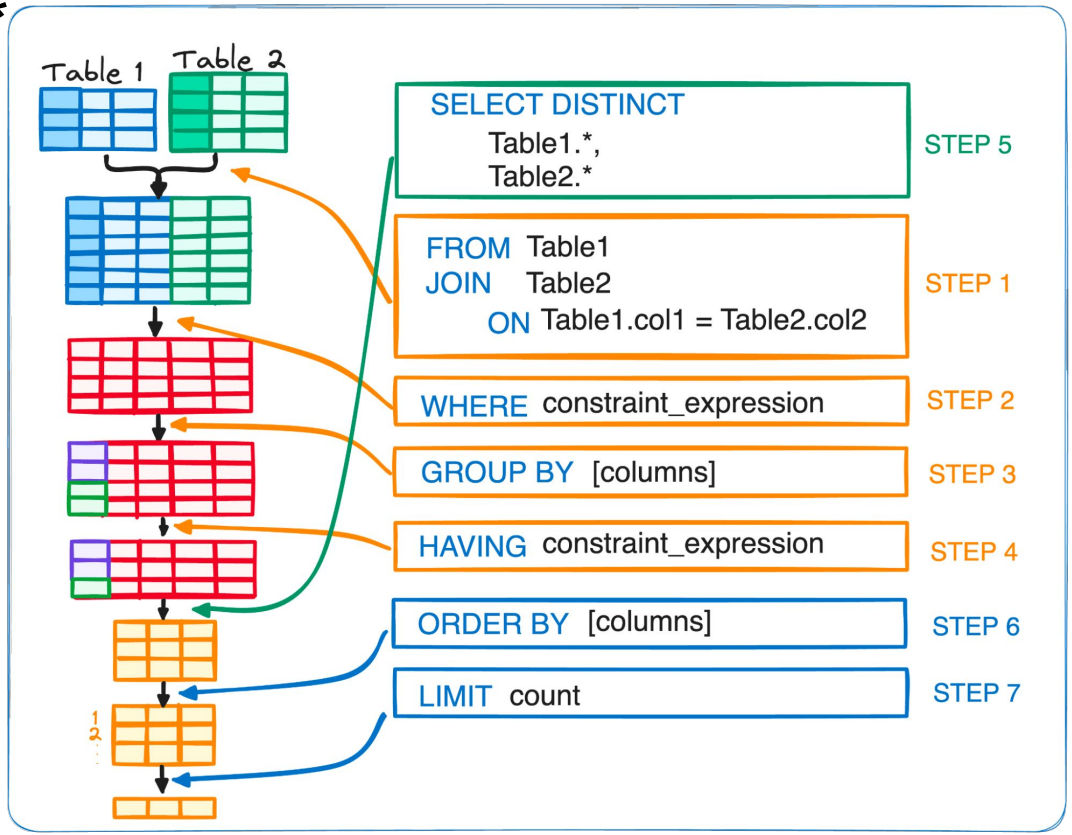
Exercise 4

Exercise 4*



Img src: https://www.thedataschool.co.uk/priya-kondola/sql-order-of-operations-3/

Exercise 4*



Img src: https://www.kdnuggets.com/the-essential-guide-to-sql-execution-order

Exercise 5.1

```
SELECT sum(l_quantity)
FROM lineitem
CROSS JOIN orders
WHERE o_orderpriority = '1-URGENT'
  AND o_orderkey = l_orderkey;
```

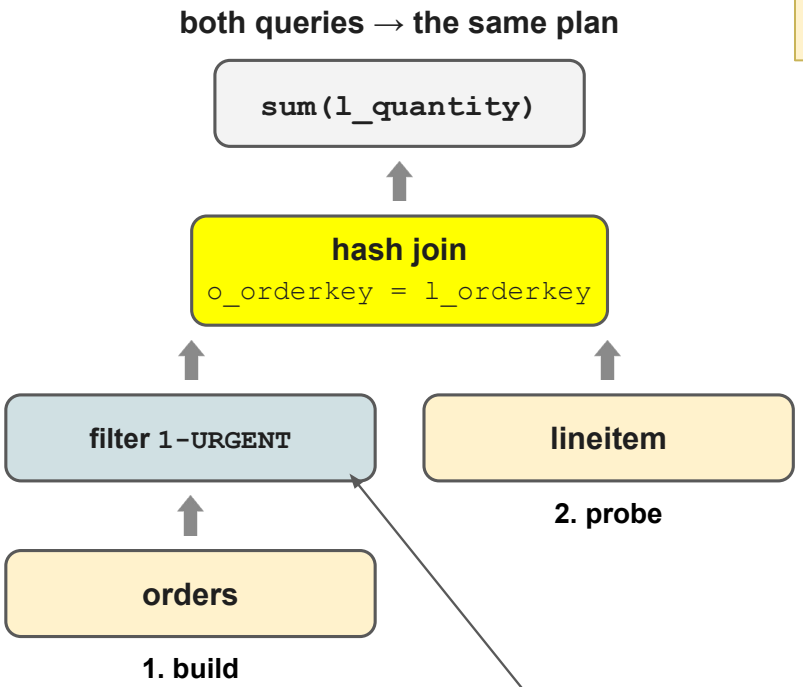
Exercise 5.2

```
SELECT sum(l_quantity)
FROM lineitem
INNER JOIN orders
  ON o_orderkey = l_orderkey
WHERE o_orderpriority = '1-URGENT';
```

Exercise 5.3

C++ and shell: we say **how** (imperative)

SQL: we say **what** (declarative)



the urgent filter runs on orders **before** the hash table is built

Thank you!

Link to slides:

